



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

CLEYTON ENRIKE DE OLIVEIRA – RM 560485 (1TDSQ)
MATHEUS HENRIQUE NASCIMENTO DE FREITAS – RM 560442 (1TDSQ)
MATHEUS PINHEIRO ERMACORA MARTIN – RM 557720 (1TDSZ)

VIASMART - PONTO DE AJUDA NAS ESTAÇÕES

SÃO PAULO 2025

CLEYTON ENRIKE DE OLIVEIRA – RM 560485 (1TDSQ)
MATHEUS HENRIQUE NASCIMENTO DE FREITAS – RM 560442 (1TDSQ)
MATHEUS PINHEIRO ERMACORA MARTIN – RM 557720 (1TDSZ)

VIASMART - PONTO DE AJUDA NAS ESTAÇÕES

Challenge apresentado pela empresa CCR, como parte do desenvolvimento das diversas áreas de conhecimento do curso de Análise e Desenvolvimento de Sistemas.

SÃO PAULO 2025

SUMÁRIO

DESAFIO DA EQUIPE: DESENVOLVER UM TOTEM DE AJUDA PARA AUXILIAR A LOCOMOÇÃO DOS USUÁRIOS NAS ESTAÇÕES.....	4
Melhorando a experiência dos usuários.....	4
Incluir para inovar e revolucionar	5
Benefícios para a Empresa.....	5
APLICAÇÃO DE JAVA NO PROJETO	6
DESCRIÇÃO DAS FUNCIONALIDADES.....	7

DESAFIO DA EQUIPE: DESENVOLVER UM TOTEM DE AJUDA PARA AUXILIAR A LOCOMOÇÃO DOS USUÁRIOS NAS ESTAÇÕES.

Objetivo da equipe: A ideia é desenvolver um serviço de atendimento personalizado nas estações, utilizando uma assistente virtual com inteligência artificial. Estamos inovando a experiência do usuário nas estações com a implementação de um totem interativo equipado com uma assistente virtual inteligente. Essa assistente será capaz de entender as necessidades individuais de cada passageiro, oferecendo informações precisas e personalizadas sobre as melhores rotas dentro do sistema metroviário. Para garantir a inclusão, o sistema oferecerá opções de idiomas, como português e inglês, e contará com recursos de acessibilidade.

Melhorando a experiência dos usuários

Queremos poder melhorar a forma como cada usuário se locomove nas estações metroviária, sendo uma tecnologia que se adapta a cada indivíduo.

Pensando em desenvolver um produto que atenda ao máximo possível de usuários. Nosso objetivo é desenvolver uma tecnologia que auxilie de maneira prática, eficiente e inclusiva a locomoção de cada usuário até a sua estação de destino.

- **Interação intuitiva:** O passageiro se aproximaria de um ponto de atendimento equipado com uma interface fácil de usar, como um totem *touch screen*.
- **Consulta personalizada:** O totem tem um menu intuitivo que permite o usuário navegar entre as funcionalidades (Consultar rota, Informações, Ajuda), sendo a principal “Consultar rota” onde ele pode ter o auxílio da rota de estações para chegar ao seu destino.
- **Orientação detalhada:** A partir das informações fornecidas, como estação de origem e destino, a assistente geraria um mapa digital ou

um roteiro passo a passo, indicando a plataforma correta e o sentido do trem.

- **Informações adicionais:** A assistente poderia fornecer informações adicionais, como horários dos trens, tempo estimado de viagem, opções de transporte público integrado e até mesmo alertas sobre possíveis interrupções no serviço.

Incluir para inovar e revolucionar

Tendo em vista a grande diversidade usuários metroviários, com diferentes necessidades, conhecimentos e nacionalidades, estamos desenvolvendo um projeto inovador que irá revolucionar a maneira como as pessoas buscam as informações dentro das estações com o intuito de incluir de forma mais abrangente todos os tipos de usuário.

Nossa principal missão é garantir que todos os usuários tenham uma experiência única e personalizada ao interagir com o nosso totem. Para podermos alcançar esse objetivo, estamos buscando implementar uma inteligência artificial que padroniza a entrada de dados, gerada pelo usuário quando faz a interação com o totem, para um tratamento e manipulação de dados mais precisa do algoritmo responsável por fazer a busca no sistema de todas as informações e apresentar dentre elas a qual satisfaz o desejo de cada usuário.

Benefícios para a Empresa

A implementação de totens informativos nas estações proporciona uma experiência personalizada e mais satisfatória para os usuários, melhorando a percepção dos usuários pelas estações, além de criar um comprometimento com a excelência no atendimento e às necessidades individuais de cada usuário.

APLICAÇÃO DE JAVA NO PROJETO

O projeto utilizou Java para a implementação de novas classes que possibilitaram o desenvolvimento de um algoritmo de Dijkstra, responsável por encontrar a melhor rota dentro do sistema metroviário. Essa abordagem permitiu a criação de um sistema eficiente para cálculo de trajetos, garantindo que os usuários recebam informações precisas e otimizadas sobre seus deslocamentos. A estrutura orientada a objetos do Java foi essencial para modularizar o código, facilitando a manutenção e escalabilidade da aplicação.

DESCRIÇÃO DAS FUNCIONALIDADES

Neste projeto temos funcionalidades x y z e sei lá o que mais

1.1 Classe UsuarioResource()

A classe UsuarioResource expõe uma API RESTful para manipulação de usuários. As funcionalidades incluem listagem de usuários, autenticação (login), cadastro, atualização de status e redefinição de senha. Utiliza JAX-RS com Jakarta RESTful Web Services para definir os endpoints e se comunica com a camada de serviço e DAO para executar as operações de negócio.

1.1.1 @GET

Listar usuários

- **Função:** Retorna todos os usuários cadastrados no sistema.
- **Lógica:** Chama o método listarUsuarios() da camada de serviço que consulta o banco via DAO e retorna uma lista de UsuarioModel.

1.1.2 @POST

Login de usuário

- **Função:** Autentica o login do usuário com base no login e senha fornecidos.
- **Lógica:**
 1. Chama o DAO para verificar se o login e senha existem no banco.
 2. Verifica se o usuário está ativo (status = "1").
 3. Retorna o objeto UsuarioModel autenticado ou uma mensagem de erro apropriada.

Cadastrar novo usuário

- **Função:** Registra um novo usuário no sistema.
- **Lógica:**
 1. Recebe o login, senha e cargo do usuário.

2. Chama a camada de serviço (cadastrarUsuario) para inserir o usuário no banco.
3. Retorna 201 Created em caso de sucesso, ou erro apropriado.

1.1.3 @PUT

Atualizar status do usuário

- **Função:** Atualiza o status (ativo ou inativo) de um usuário pelo ID.
- **Lógica:**
 1. Recebe o ID do usuário e o novo status como texto puro.
 2. Chama atualizarStatus da camada de serviço.
 3. Retorna 200 OK se o status foi alterado, ou 404 Not Found se o usuário não existir.

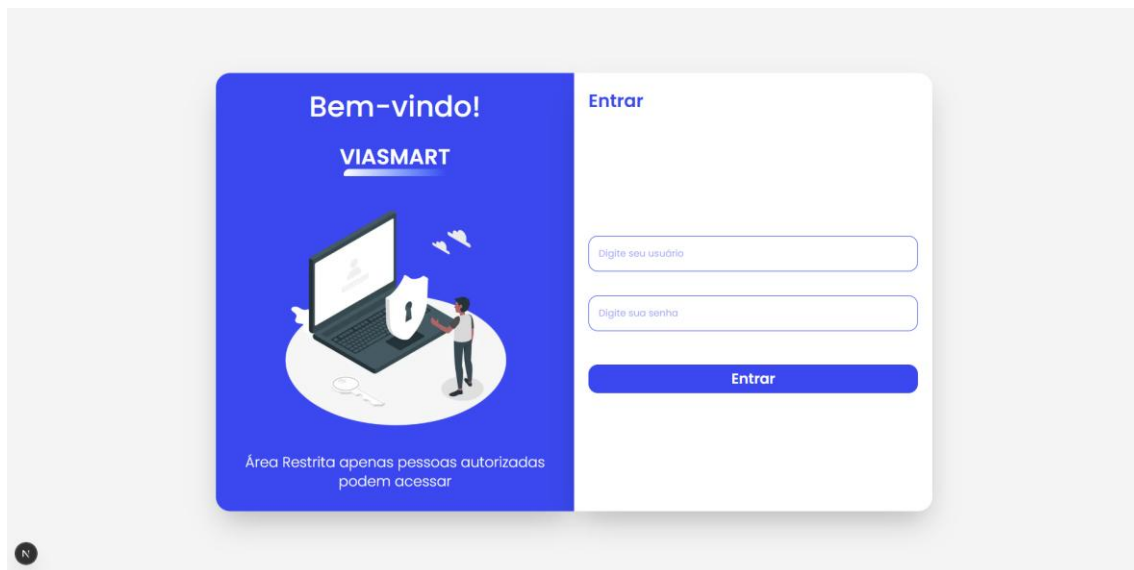
Redefinir senha de um usuário

- **Função:** Permite alterar a senha de um usuário específico.
- **Lógica:**
 1. Recebe o ID do usuário e a nova senha como texto.
 2. Chama o DAO (atualizarSenha) para alterar no banco.
 3. Retorna 200 OK se a senha foi alterada com sucesso ou erro correspondente.


```

1 package br.com.fiap.resource;
2
3 import br.com.fiap.dao.UsuarioDAO;
4 import br.com.fiap.model.LoginResponse;
5 import br.com.fiap.model.UsuarioModel;
6 import br.com.fiap.service.UsuarioService;
7
8 import jakarta.ws.rs.*;
9 import jakarta.ws.rs.core.MediaType;
10 import jakarta.ws.rs.core.Response;
11
12 import java.sql.SQLException;
13 import java.util.List;
14
15 @Path("/usuarios")
16 @Consumes(MediaType.APPLICATION_JSON)
17 @Produces(MediaType.APPLICATION_JSON)
18 public class UsuarioResource {
19
20     UsuarioService service = new UsuarioService();
21
22     @GET
23     public List<UsuarioModel> listar() {
24         try {
25             return service.listarUsuarios();
26         } catch (Exception e) {
27             throw new RuntimeException("Erro ao listar usuários: " + e.getMessage());
28         }
29     }
30
31     @POST
32     @Path("/login")
33     @Consumes(MediaType.APPLICATION_JSON)
34     @Produces(MediaType.APPLICATION_JSON)
35     public Response login(UsuarioModel loginRequest) {
36         try {
37             UsuarioModel usuario = new UsuarioDAO().autenticar(
38                 loginRequest.getLogin(), loginRequest.getSenha());
39
40             if (usuario == null) {
41                 return Response.status(Response.Status.UNAUTHORIZED)
42                     .entity("❌ Login ou senha inválidos.").build();
43             }
44
45             if (!"1".equals(usuario.getStatus())) {
46                 return Response.status(Response.Status.FORBIDDEN)
47                     .entity("❌ Seu acesso está pausado. Contate o administrador.").build();
48             }
49
50             return Response.ok(usuario).build();
51         } catch (Exception e) {
52             return Response.serverError().entity("Erro ao processar login: " + e.getMessage()).build();
53         }
54     }
55
56     @POST
57     @Path("/cadastrar")
58     public Response cadastrarUsuario(UsuarioModel usuario) {
59         try {
60             boolean sucesso = service.cadastrarUsuario(
61                 usuario.getLogin(),
62                 usuario.getSenha(),
63                 usuario.getCargo()
64             );
65
66             if (sucesso) {
67                 return Response.status(Response.Status.CREATED)
68                     .entity("✅ Usuário cadastrado com sucesso!").build();
69             } else {
70                 return Response.status(Response.Status.BAD_REQUEST)
71                     .entity("❌ Falha ao cadastrar o usuário.").build();
72             }
73         } catch (Exception e) {
74             return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
75                 .entity("Erro ao cadastrar: " + e.getMessage()).build();
76         }
77     }
78
79     @PUT
80     @Path("/{id}/status")
81     @Consumes(MediaType.TEXT_PLAIN)
82     public Response atualizarStatus(@PathParam("id") Long id, String novoStatus) {
83         try {
84             boolean atualizado = service.atualizarStatus(id, novoStatus);
85
86             if (atualizado) {
87                 return Response.ok("Status atualizado com sucesso.").build();
88             } else {
89                 return Response.status(Response.Status.NOT_FOUND)
90                     .entity("Usuário não encontrado.").build();
91             }
92         } catch (Exception e) {
93             return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
94                 .entity("Erro ao atualizar status: " + e.getMessage()).build();
95         }
96     }
97
98     @PUT
99     @Path("/{id}/senha")
100     @Consumes(MediaType.TEXT_PLAIN)
101     public Response redefinirSenha(@PathParam("id") Long id, String novaSenha) {
102         try {
103             boolean atualizado = new UsuarioDAO().atualizarSenha(id, novaSenha.trim());
104
105             if (atualizado) {
106                 return Response.ok("✅ Senha atualizada com sucesso").build();
107             } else {
108                 return Response.status(Response.Status.NOT_FOUND).entity("❌ Usuário não encontrado").build();
109             }
110         } catch (Exception e) {
111             return Response.serverError().entity("Erro ao atualizar senha: " + e.getMessage()).build();
112         }
113     }
114 }
115
116 }

```



O usuário deverá digitar um login válido para poder ser autenticado, quando clicar no botão irá fazer uma requisição na API que verificará se o login existe no banco de dados para permitir acesso as demais páginas.

1.2 Classe RotaResource()

O recurso permite que o cliente envie parâmetros de origem e destino por meio de uma requisição HTTP GET e receba como resposta o caminho entre esses pontos, se existir.

1.2.1 @GET

Calcular rota entre duas estações

- **Função:** Retorna o caminho entre uma estação de origem e uma de destino fornecidas como parâmetros.
- **Lógica:**
 1. Verifica se os parâmetros origem e destino foram informados corretamente.
 2. Se algum dos parâmetros estiver ausente ou vazio, retorna erro 400 com mensagem apropriada.
 3. Chama o serviço RotaService para calcular o caminho entre as estações.
 4. Se não houver rota possível, retorna erro 404 com mensagem de rota não encontrada.
 5. Caso a rota seja encontrada, retorna uma lista com as estações do caminho no formato JSON.
 6. Em caso de erro interno, retorna erro 500 com mensagem padrão.

```
1 package br.com.fiap.resource;
2
3 import br.com.fiap.service.RotaService;
4 import jakarta.ws.rs.*;
5 import jakarta.ws.rs.core.MediaType;
6 import jakarta.ws.rs.core.Response;
7
8 import java.util.List;
9
10 @Path("/rota")
11 @Produces(MediaType.APPLICATION_JSON)
12 public class RotaResource {
13
14     @GET
15     public Response calcularRota(@QueryParam("origem") String origem, @QueryParam("destino") String destino) {
16         try {
17             if (origem == null || destino == null || origem.trim().isEmpty() || destino.trim().isEmpty()) {
18                 return Response.status(Response.Status.BAD_REQUEST)
19                     .entity("{\"erro\": \"Origem e destino são obrigatórios.\"}")
20                     .build();
21             }
22
23             RotaService service = new RotaService();
24             List<String> caminho = service.calcularRota(origem, destino);
25
26             if (caminho.isEmpty()) {
27                 return Response.status(Response.Status.NOT_FOUND)
28                     .entity("{\"erro\": \"Caminho não encontrado entre as estações.\"}")
29                     .build();
30             }
31
32             return Response.ok(caminho).build(); // retorno automático como JSON
33         } catch (Exception e) {
34             e.printStackTrace();
35             return Response.serverError()
36                 .entity("{\"erro\": \"Erro interno ao calcular a rota.\"}")
37                 .build();
38         }
39     }
40 }
41
```

1.3 Classe LinhaResource()

O recurso permite que o cliente acesse uma lista de linhas disponíveis por meio de uma requisição HTTP GET, retornando os dados em formato JSON.

1.3.1 @GET

Listar todas as linhas disponíveis

- **Função:** Retorna uma lista com todas as linhas cadastradas no sistema.
- **Lógica:**
 1. Inicia uma instância do serviço LinhaService.
 2. Chama o método listarLinhas() da camada de serviço para buscar os dados.
 3. Caso a operação ocorra com sucesso, retorna a lista de objetos LinhaModel em formato JSON.
 4. Se ocorrer algum erro durante a busca, lança uma exceção genérica com mensagem de erro apropriada.

```
1 package br.com.fiap.resource;
2
3 import br.com.fiap.model.LinhaModel;
4 import br.com.fiap.service.LinhaService;
5
6 import jakarta.ws.rs.GET;
7 import jakarta.ws.rs.Path;
8 import jakarta.ws.rs.Produces;
9 import jakarta.ws.rs.core.MediaType;
10
11 import java.util.List;
12
13 @Path("/linhas")
14 public class LinhaResource {
15     LinhaService service = new LinhaService();
16
17     @GET
18     @Produces(MediaType.APPLICATION_JSON)
19     public List<LinhaModel> listar() {
20         try {
21             return service.listarLinhas();
22         } catch (Exception e) {
23             throw new RuntimeException("Erro ao listar linhas: " + e.getMessage());
24         }
25     }
26 }
```

CCR

ViaSmart

Filtros

Normal

Pausada

Limpar

Linhas e Estações

Linha	Estação	Ação	Status
Linha 1 – Azul	Ana Rosa		Ativa
Linha 1 – Azul	Armênia		Ativa
Linha 1 – Azul	Carandiru		Ativa
Linha 1 – Azul	Conceição		Ativa
Linha 1 – Azul	Jabaquara		Ativa
Linha 1 – Azul	Japão – Liberdade		Ativa
Linha 1 – Azul	Jardim São Paulo – Ayrton Senna		Ativa
Linha 1 – Azul	Luz		Ativa
Linha 1 – Azul	Parada Inglesa		Ativa
Linha 1 – Azul	Paraíso		Ativa
Linha 1 – Azul	Portuguesa – Tietê		Ativa
Linha 1 – Azul	Praça da Árvore		Ativa

N

Responsável por listar as linhas contidas no banco de dados

1.4 Classe IdiomaResource()

O recurso permite que o cliente acesse uma lista de idiomas disponíveis por meio de uma requisição HTTP GET, retornando os dados em formato JSON.

1.4.1 @GET

Listar todos os idiomas disponíveis

- **Função:** Retorna uma lista com todos os idiomas cadastrados no sistema.
- **Lógica:**

1. Inicia uma instância da classe de serviço IdiomaService.
2. Chama o método listarIdiomas() da camada de serviço para obter a lista de idiomas.
3. Caso a operação seja bem-sucedida, retorna a lista de objetos IdiomaModel em formato JSON.
4. Se ocorrer alguma exceção durante a execução, lança uma exceção genérica com uma mensagem informando o erro ocorrido.

```
1 package br.com.fiap.resource;
2
3 import br.com.fiap.model.IdiomaModel;
4 import br.com.fiap.service.IdiomaService;
5
6 import jakarta.ws.rs.GET;
7 import jakarta.ws.rs.Path;
8 import jakarta.ws.rs.Produces;
9 import jakarta.ws.rs.core.MediaType;
10
11 import java.util.List;
12
13 @Path("/idiomas")
14 public class IdiomaResource {
15     IdiomaService service = new IdiomaService();
16
17     @GET
18     @Produces(MediaType.APPLICATION_JSON)
19     public List<IdiomaModel> listar() {
20         try {
21             return service.listarIdiomas();
22         } catch (Exception e) {
23             throw new RuntimeException("Erro ao listar idiomas: " + e.getMessage());
24         }
25     }
26 }
```

1.5 Classe EstacaoResource()

Este recurso expõe endpoints REST relacionados à entidade **Estação**, permitindo que o cliente consulte todas as estações, atualize o status de uma estação específica e acesse apenas os nomes das estações. Os dados são trafegados em formato JSON (exceto na atualização de status, que consome texto puro).

1.5.1 @GET

Listar todas as estações

- **Função:** Retorna uma lista com todas as estações cadastradas no sistema.
- **Lógica:**
 1. Cria uma instância da classe de serviço EstacaoService.
 2. Chama o método listarEstacoes() da camada de serviço.
 3. Retorna a lista de objetos EstacaoModel em formato JSON.
 4. Em caso de erro, lança uma exceção genérica com mensagem explicativa.

Listar somente os nomes das estações

- **Função:** Retorna uma lista com apenas os nomes das estações cadastradas.
- **Lógica:**
 1. Cria uma nova instância da classe EstacaoService.
 2. Chama o método listarNomesEstacoes() da camada de serviço.
 3. Retorna a lista de nomes como uma List<String> em formato JSON.
 4. Em caso de erro, retorna HTTP 500 com a mensagem de erro.

@PUT

Atualizar o status de uma estação específica

- **Função:** Atualiza o status de uma estação identificada pelo ID informado na URL.
- **Lógica:**
 1. Recebe o id da estação via parâmetro de caminho e o novo status como corpo da requisição (texto puro).
 2. Utiliza a classe DAO (EstacaoDAO) para atualizar o status no banco de dados por meio do método atualizarStatus().
 3. Se a atualização for bem-sucedida, retorna uma resposta HTTP 200 com mensagem de sucesso.
 4. Se a estação não for encontrada, retorna HTTP 404 com mensagem adequada.
 5. Em caso de erro, retorna HTTP 500 com a mensagem de erro.

```

1  package br.com.fiap.resource;
2
3
4  import br.com.fiap.dao.EstacaoDAO;
5  import br.com.fiap.model.EstacaoModel;
6  import br.com.fiap.service.EstacaoService;
7
8  import jakarta.ws.rs.*;
9  import jakarta.ws.rs.core.MediaType;
10 import jakarta.ws.rs.core.Response;
11
12 import java.util.List;
13
14 @Path("/estacoes")
15 public class EstacaoResource {
16     EstacaoService service = new EstacaoService();
17
18     @GET
19     @Produces(MediaType.APPLICATION_JSON)
20     public List<EstacaoModel> listar() {
21         try {
22             return service.listarEstacoes();
23         } catch (Exception e) {
24             throw new RuntimeException("Erro ao listar estações: " + e.getMessage());
25         }
26     }
27
28     @PUT
29     @Path("/{id}/status")
30     @Consumes(MediaType.TEXT_PLAIN)
31     public Response atualizarStatusEstacao(@PathParam("id") Long id, String novoStatus) {
32         try {
33             boolean atualizado = new EstacaoDAO().atualizarStatus(id, novoStatus);
34             if (atualizado) {
35                 return Response.ok("✅ Status atualizado com sucesso").build();
36             } else {
37                 return Response.status(Response.Status.NOT_FOUND)
38                     .entity("❌ Estação não encontrada").build();
39             }
40         } catch (Exception e) {
41             return Response.serverError().entity("Erro ao atualizar: " + e.getMessage()).build();
42         }
43     }
44
45     @GET
46     @Path("/nomes")
47     @Produces(MediaType.APPLICATION_JSON)
48     public Response listarSomenteNomes() {
49         try {
50             List<String> estacoes = new EstacaoService().listarNomesEstacoes();
51             return Response.ok(estacoes).build();
52         } catch (Exception e) {
53             return Response.serverError().entity("Erro ao buscar estações: " + e.getMessage()).build();
54         }
55     }
56 }
57
58 }
59
60

```

CCR

ViaSmart

Filtros

Normal

Pausada

Limpar

Linhas e Estações

Linha	Estação	Ação	Status
Linha 1 – Azul	Ana Rosa	☒	Ativa
Linha 1 – Azul	Armênia	☒	Ativa
Linha 1 – Azul	Carandiru	☒	Ativa
Linha 1 – Azul	Conceição	☒	Ativa
Linha 1 – Azul	Jabaquara	☒	Ativa
Linha 1 – Azul	Japão – Liberdade	☒	Ativa
Linha 1 – Azul	Jardim São Paulo – Ayrton Senna	☒	Ativa
Linha 1 – Azul	Luz	☒	Ativa
Linha 1 – Azul	Parada Inglesa	☒	Ativa
Linha 1 – Azul	Paraíso	☒	Ativa
Linha 1 – Azul	Portuguesa – Tietê	☒	Ativa
Linha 1 – Azul	Praça da Árvore	☒	Ativa

N

Responsável por listar as estações e status de cada uma, contidas no banco de dados

1.6 Classe EstacaoLinhaResource()

1.6.1 @GET

Listar estações com suas respectivas linhas

- **Função:** Retorna uma lista de objetos LinhaEstacaoDTO, que representam as estações e as linhas às quais estão associadas.
- **Lógica:**
 1. Cria uma instância da classe DAO EstacaoLinhaDAO.
 2. Chama o método listarEstacoesComLinhas() da camada DAO.
 3. Retorna a lista de objetos LinhaEstacaoDTO em formato JSON.
 4. Em caso de falha na consulta (por exemplo, problemas com o banco de dados), lança uma WebApplicationException com status 500 e uma mensagem de erro explicativa.

```
1  package br.com.fiap.resource;
2
3  import br.com.fiap.dao.EstacaoLinhaDAO;
4  import br.com.fiap.dao.LinhaDAO;
5  import br.com.fiap.dto.LinhaEstacaoDTO;
6  import br.com.fiap.model.EstacaoLinhaModel;
7  import br.com.fiap.service.EstacaoLinhaService;
8
9  import jakarta.ws.rs.GET;
10 import jakarta.ws.rs.Path;
11 import jakarta.ws.rs.Produces;
12 import jakarta.ws.rs.WebApplicationException;
13 import jakarta.ws.rs.core.MediaType;
14
15 import java.sql.SQLException;
16 import java.util.List;
17 @Path("/estacoes-linhas")
18 @Produces(MediaType.APPLICATION_JSON)
19 public class EstacaoLinhaResource {
20
21     private EstacaoLinhaDAO dao = new EstacaoLinhaDAO();
22
23     @GET
24     public List<LinhaEstacaoDTO> listar() {
25         try {
26             return dao.listarEstacoesComLinhas();
27         } catch (SQLException e) {
28             throw new WebApplicationException("Erro ao buscar estações", 500);
29         }
30     }
31 }
```

CCR | ViaSmart

Filtros

Normal

Pausada

Limpar

Linha e Estações

Linha	Estação	Ação	Status
Linha 1 – Azul	Ana Rosa	☒	Ativa
Linha 1 – Azul	Armênia	☒	Ativa
Linha 1 – Azul	Carandiru	☒	Ativa
Linha 1 – Azul	Conceição	☒	Ativa
Linha 1 – Azul	Jabaquara	☒	Ativa
Linha 1 – Azul	Japão – Liberdade	☒	Ativa
Linha 1 – Azul	Jardim São Paulo – Ayrton Senna	☒	Ativa
Linha 1 – Azul	Luz	☒	Ativa
Linha 1 – Azul	Parada Inglesa	☒	Ativa
Linha 1 – Azul	Paraíso	☒	Ativa
Linha 1 – Azul	Portuguesa – Tietê	☒	Ativa
Linha 1 – Azul	Praça da Árvore	☒	Ativa

Responsável por listar as linhas e as estações e status de cada uma, contidas no banco de dados

1.7 Classe DuvidaFrequenteResource()

1.7.1 @GET

Listar dúvidas frequentes

- **Função:** Retorna uma lista de dúvidas frequentes armazenadas no sistema.
- **Lógica:**
 1. Cria uma instância do serviço DuvidaFrequenteService.
 2. Chama o método listarDuvidas() que busca todas as dúvidas.
 3. Retorna a lista de objetos DuvidaFrequenteModel em formato JSON.
 4. Em caso de erro, lança uma exceção genérica com uma mensagem explicativa.

1.7.2 @POST

Cadastrar nova dúvida frequente

- **Função:** Registra uma nova dúvida frequente no sistema.
- **Lógica:**
 1. Recebe um objeto DuvidaFrequenteModel no corpo da requisição.
 2. Chama o método cadastrarDuvida(duvida) da camada de serviço.
 3. Se o cadastro for bem-sucedido, retorna status 201 com mensagem de sucesso.
 4. Se houver falha na validação ou no processo, retorna status 400.
 5. Em caso de erro interno, retorna status 500 com mensagem explicativa.

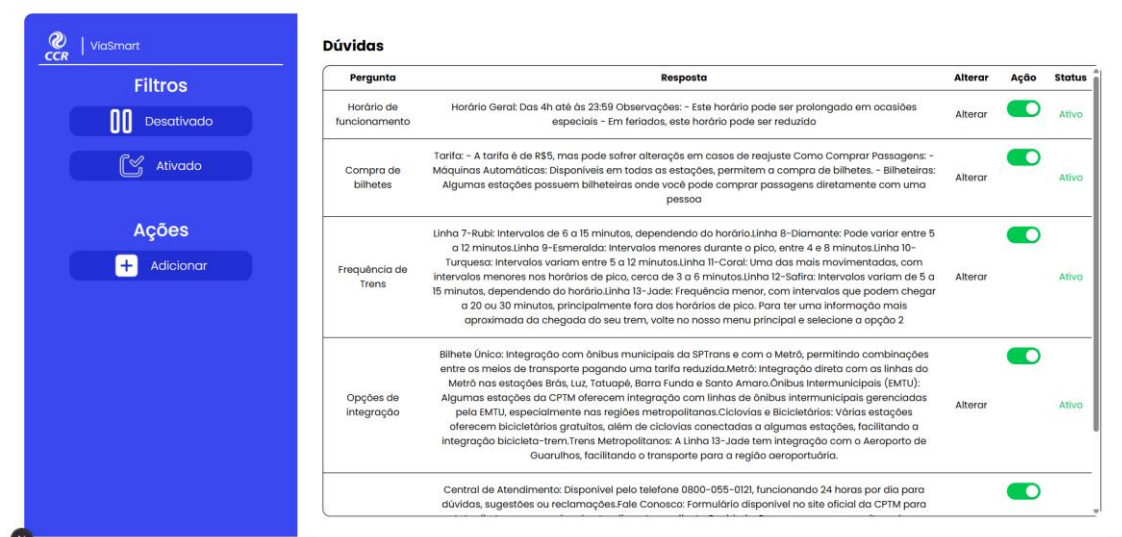
1.7.3 @PUT

Atualizar uma dúvida frequente existente

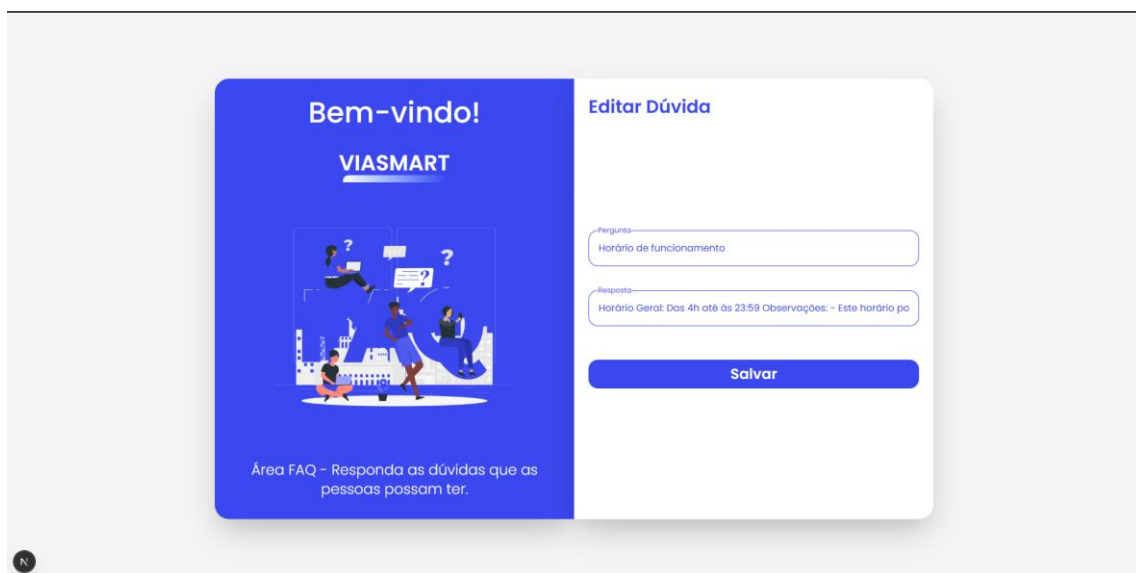
- **Função:** Atualiza os dados de uma dúvida específica com base no ID informado.
- **Lógica:**
 1. Recebe o id via path param e o objeto DuvidaFrequenteModel com as novas informações.
 2. Chama o método atualizarDuvida(id, duvida) da camada de serviço.
 3. Se a dúvida for encontrada e atualizada, retorna status 200 com mensagem de sucesso.

4. Se a dúvida não existir, retorna status 404.
5. Em caso de erro interno, retorna status 500 com mensagem de erro.

```
1 package br.com.fiap.resource;
2
3 import br.com.fiap.model.DuvidaFrequenteModel;
4 import br.com.fiap.service.DuvidaFrequenteService;
5
6 import jakarta.ws.rs.*;
7 import jakarta.ws.rs.core.MediaType;
8 import jakarta.ws.rs.core.Response;
9 import java.util.List;
10
11 @Path("/duvidas")
12 @Consumes(MediaType.APPLICATION_JSON)
13 @Produces(MediaType.APPLICATION_JSON)
14 public class DuvidaFrequenteResource {
15
16     DuvidaFrequenteService service = new DuvidaFrequenteService();
17
18     @GET
19     public List<DuvidaFrequenteModel> listar() {
20         try {
21             return service.listarDuvidas();
22         } catch (Exception e) {
23             throw new RuntimeException("Erro ao listar dúvidas frequentes: " + e.getMessage());
24         }
25     }
26
27     @PUT
28     @Path("/{id}/status")
29     @Consumes(MediaType.TEXT_PLAIN)
30     public Response atualizarStatus(@PathParam("id") Long id, String novoStatus) {
31         try {
32             boolean sucesso = service.atualizarStatus(id, novoStatus.trim());
33             if (sucesso) {
34                 return Response.ok("✅ Status atualizado com sucesso").build();
35             } else {
36                 return Response.status(Response.Status.NOT_FOUND)
37                     .entity("❌ Dúvida não encontrada").build();
38             }
39         } catch (Exception e) {
40             return Response.serverError().entity("Erro ao atualizar status: " + e.getMessage()).build();
41         }
42     }
43
44     @POST
45     public Response cadastrarDuvida(DuvidaFrequenteModel duvida) {
46         try {
47             boolean sucesso = service.cadastrarDuvida(duvida);
48             if (sucesso) {
49                 return Response.status(Response.Status.CREATED)
50                     .entity("✅ Dúvida cadastrada com sucesso").build();
51             } else {
52                 return Response.status(Response.Status.BAD_REQUEST)
53                     .entity("❌ Falha ao cadastrar a dúvida").build();
54             }
55         } catch (Exception e) {
56             return Response.serverError().entity("Erro interno: " + e.getMessage()).build();
57         }
58     }
59
60     @PUT
61     @Path("/{id}")
62     @Consumes(MediaType.APPLICATION_JSON)
63     @Produces(MediaType.APPLICATION_JSON)
64     public Response atualizarDuvida(@PathParam("id") Long id, DuvidaFrequenteModel duvida) {
65         try {
66             boolean sucesso = new DuvidaFrequenteService().atualizarDuvida(id, duvida);
67             if (sucesso) {
68                 return Response.ok("✅ Dúvida atualizada com sucesso").build();
69             } else {
70                 return Response.status(Response.Status.NOT_FOUND).entity("❌ Dúvida não encontrada").build();
71             }
72         } catch (Exception e) {
73             return Response.serverError().entity("❌ Erro ao atualizar: " + e.getMessage()).build();
74         }
75     }
76
77 }
```



Responsável por listas as perguntas, respostas e status e Também fazer atualização no status



Responsável por fazer uma atualização em alguma dúvida específica

Bem-vindo!

VIASMART



Área FAQ - Responda as dúvidas que as pessoas possam ter.

Cadastrar Dúvida

Salvar

Responsável por adicionar uma nova dúvida no banco de dados

1.8 Classe BuscaEstacaoResource()

1.8.1 @GET

Listar todas as buscas de estações

- **Função:** Retorna uma lista com todas as buscas de estações armazenadas.

- **Lógica:**

1. Cria uma instância da classe de serviço BuscaEstacaoService.
2. Chama o método listarBuscas() da camada de serviço.
3. Retorna a lista de objetos BuscaEstacaoModel em formato JSON.
4. Em caso de erro, captura a exceção e lança uma RuntimeException com uma mensagem explicativa.

```
1  package br.com.fiap.resource;
2
3  import br.com.fiap.model.BuscaEstacaoModel;
4  import br.com.fiap.service.BuscaEstacaoService;
5
6  import jakarta.ws.rs.GET;
7  import jakarta.ws.rs.Path;
8  import jakarta.ws.rs.Produces;
9  import jakarta.ws.rs.core.MediaType;
10
11 import java.util.List;
12
13 @Path("/buscas-estacoes")
14 public class BuscaEstacaoResource {
15     BuscaEstacaoService service = new BuscaEstacaoService();
16
17     @GET
18     @Produces(MediaType.APPLICATION_JSON)
19     public List<BuscaEstacaoModel> listar() {
20         try {
21             return service.listarBuscas();
22         } catch (Exception e) {
23             throw new RuntimeException("Erro ao listar buscas de estações: " + e.getMessage());
24         }
25     }
26 }
```

1.9 Classe Grafo()

```
1 // Classe Grafo para representar o mapa
2 public static class Grafo {
3     private final Map<String, List<String>> adjacencia;
4
5     public Grafo() {
6         adjacencia = new HashMap<>();
7     }
8
9     public void addEdge(String origem, String destino, int peso) {
10         adjacencia.putIfAbsent(origem, new ArrayList<>());
11         adjacencia.get(origem).add(destino);
12     }
13
14     public List<String> caminhoMaisCurto(String origem, String destino) {
15         // Algoritmo de busca em largura (BFS) para encontrar o caminho mais curto
16         Queue<String> fila = new LinkedList<>();
17         Map<String, String> predecessores = new HashMap<>();
18         Set<String> visitados = new HashSet<>();
19
20         fila.add(origem);
21         visitados.add(origem);
22
23         while (!fila.isEmpty()) {
24             String atual = fila.poll();
25
26             if (atual.equals(destino)) {
27                 List<String> caminho = new ArrayList<>();
28                 for (String station = destino; station != null; station = predecessores.get(station)) {
29                     caminho.add(station);
30                 }
31                 Collections.reverse(caminho);
32                 return caminho;
33             }
34
35             for (String vizinho : adjacencia.getOrDefault(atual, new ArrayList<>())) {
36                 if (!visitados.contains(vizinho)) {
37                     fila.add(vizinho);
38                     visitados.add(vizinho);
39                     predecessores.put(vizinho, atual);
40                 }
41             }
42         }
43         return Collections.emptyList(); // Se não encontrar o caminho
44     }
45 }
```

A classe Grafo é usada para representar o mapa das estações e suas conexões. Ela utiliza uma estrutura de dados do tipo Map para armazenar as adjacências entre as estações e implementa um algoritmo de busca em largura (BFS) para encontrar o caminho mais curto entre duas estações.

Funcionalidade:

1. Adição de arestas: O método `addEdge` adiciona uma conexão entre duas estações.
2. Busca do caminho mais curto: O método `caminhoMaisCurto` utiliza o algoritmo BFS para encontrar o menor caminho entre a origem e o destino.

Passos do algoritmo:

1. Inicializa uma fila para armazenar as estações a serem visitadas.
2. Usa um Map para rastrear os predecessores de cada estação.
3. Marca as estações visitadas para evitar loops.
4. Reconstrói o caminho a partir do destino até a origem, invertendo-o no final.

1.10 Classe Caminho()

```
1 public static void consultarCaminho() {
2     String origem = JOptionPane.showInputDialog(null, "Qual estação você está?");
3     String destino = JOptionPane.showInputDialog(null, "Qual estação você quer ir?");
4
5     // Criar o grafo e adicionar as arestas
6     Grafo grafo = new Grafo();
7     grafo.addEdge("Júlio Prestes", "Palmeiras-Barra Funda", 1);
8     grafo.addEdge("Palmeiras-Barra Funda", "Júlio Prestes", 1);
9     grafo.addEdge("Palmeiras-Barra Funda", "Lapa", 1);
10    grafo.addEdge("Lapa", "Palmeiras-Barra Funda", 1);
11    grafo.addEdge("Lapa", "Domingos de Moraes", 1);
12    grafo.addEdge("Domingos de Moraes", "Lapa", 1);
13    grafo.addEdge("Domingos de Moraes", "Imperatriz Leopoldina", 1);
14    grafo.addEdge("Imperatriz Leopoldina", "Domingos de Moraes", 1);
15    grafo.addEdge("Imperatriz Leopoldina", "Presidente Altino", 1);
16    grafo.addEdge("Presidente Altino", "Imperatriz Leopoldina", 1);
17    grafo.addEdge("Presidente Altino", "Osasco", 1);
18    grafo.addEdge("Osasco", "Presidente Altino", 1);
19    grafo.addEdge("Osasco", "Comandante Sampaio", 1);
20    grafo.addEdge("Comandante Sampaio", "Osasco", 1);
21    grafo.addEdge("Comandante Sampaio", "Quitaúna", 1);
22    grafo.addEdge("Quitaúna", "Comandante Sampaio", 1);
23    grafo.addEdge("Quitaúna", "General Miguel Costa", 1);
24    grafo.addEdge("General Miguel Costa", "Quitaúna", 1);
25    grafo.addEdge("General Miguel Costa", "Carapicuíba", 1);
26    grafo.addEdge("Carapicuíba", "General Miguel Costa", 1);
27    grafo.addEdge("Carapicuíba", "Santa Terezinha", 1);
28    grafo.addEdge("Santa Terezinha", "Carapicuíba", 1);
29    grafo.addEdge("Santa Terezinha", "Antonio João", 1);
30    grafo.addEdge("Antonio João", "Santa Terezinha", 1);
31    grafo.addEdge("Antonio João", "Barueri", 1);
32    grafo.addEdge("Barueri", "Antonio João", 1);
33    grafo.addEdge("Barueri", "Jardim Belval", 1);
34    grafo.addEdge("Jardim Belval", "Barueri", 1);
35    grafo.addEdge("Jardim Belval", "Jardim Silveira", 1);
36    grafo.addEdge("Jardim Silveira", "Jardim Belval", 1);
37    grafo.addEdge("Jardim Silveira", "Jandira", 1);
38    grafo.addEdge("Jandira", "Jardim Silveira", 1);
39    grafo.addEdge("Jandira", "Sagrado Coração", 1);
40    grafo.addEdge("Sagrado Coração", "Jandira", 1);
41    grafo.addEdge("Sagrado Coração", "Engenheiro Cardoso", 1);
42    grafo.addEdge("Engenheiro Cardoso", "Sagrado Coração", 1);
43    grafo.addEdge("Engenheiro Cardoso", "Itapevi", 1);
44    grafo.addEdge("Itapevi", "Engenheiro Cardoso", 1);
45    grafo.addEdge("Itapevi", "Amador Bueno", 1);
46    grafo.addEdge("Amador Bueno", "Itapevi", 1);
47
48    // Encontrar o caminho mais curto
49    List<String> caminhoMaisCurto = grafo.caminhoMaisCurto(origem, destino);
50
51    // Exibir o caminho
52    if (caminhoMaisCurto.isEmpty()) {
53        JOptionPane.showMessageDialog(null, "Não foi possível encontrar o caminho entre as estações.");
54    } else {
55        JOptionPane.showMessageDialog(null, "Caminho mais curto: " + caminhoMaisCurto);
56    }
57 }
```

Este método é responsável por interagir com o usuário, coletar as estações de origem e destino, e exibir o caminho mais curto entre elas.

Funcionalidade:

- Entrada do usuário: Usa JOptionPane para solicitar ao usuário a estação de origem e destino.
- Construção do grafo: Adiciona manualmente as conexões entre as estações.
- Busca e exibição do caminho: Chama o método caminhoMaisCurto da classe Grafo e exibe o resultado em uma caixa de diálogo.

1. Resultado:

- Se o caminho for encontrado, ele é retornado como uma lista de estações.
- Caso contrário, uma lista vazia é retornada.

4.Reconstrução do Caminho:

- Após encontrar o destino, o caminho é reconstruído a partir do destino até a origem, utilizando o mapa de predecessores.
- O caminho é invertido para exibir a rota na ordem correta (origem → destino).

5.Resultado:

- Se o caminho for encontrado, ele é retornado como uma lista de estações.
- Caso contrário, uma lista vazia é retornada.

1.Tela Menu



```
1 public static void menuPrincipal() {
2     String opcao = JOptionPane.showInputDialog(null, "Menu Principal: 1 - Consultar caminho mais curto; 2 - Informações sobre lotação de trem; 3 - Dividas frequentes; 4 - Cadastrar linha; 5 - Encerrar atendimento", "Menu", JOptionPane.QUESTION_MESSAGE);
3
4     while ((opcao.matches("[1-5]"))) {
5         opcao = JOptionPane.showInputDialog(null, "Opção inválida. Escolha uma opção entre 1 e 5.", "Erro", JOptionPane.ERROR_MESSAGE);
6     }
7
8     switch (opcao) {
9         case "1":
10             Caminho.consultarCaminho(); // Chama o método na classe Caminho
11             break;
12         case "2":
13             Lotacao.mostrarLotacao(); // Chama o método na classe Lotacao
14             break;
15         case "3":
16             DividasFrequentes.mostrarDividasFrequentes(); // Chama o método na classe DividasFrequentes
17             break;
18         case "4":
19             try {
20                 InsertLinha.insert(); // Chama o método para inserir linha
21             } catch (SQLException e) {
22                 System.out.println(e.getMessage());
23             }
24             break;
25         case "5":
26             encerrar(); // Chama o método para encerrar o programa
27             break;
28     }
29 }
```

A tela menu permite que o usuário navegue pelo sistema, podendo escolher qual funcionalidade ele deseja consultar. No caso do projeto, a principal funcionalidade é a “Consultar Rota”.

2.Tela Consultar Rota

Voltar

Consultar rota

Sua estação atual

Vila Mariana

Sua estação final

Bresser - Mooca

Mostrar Rota

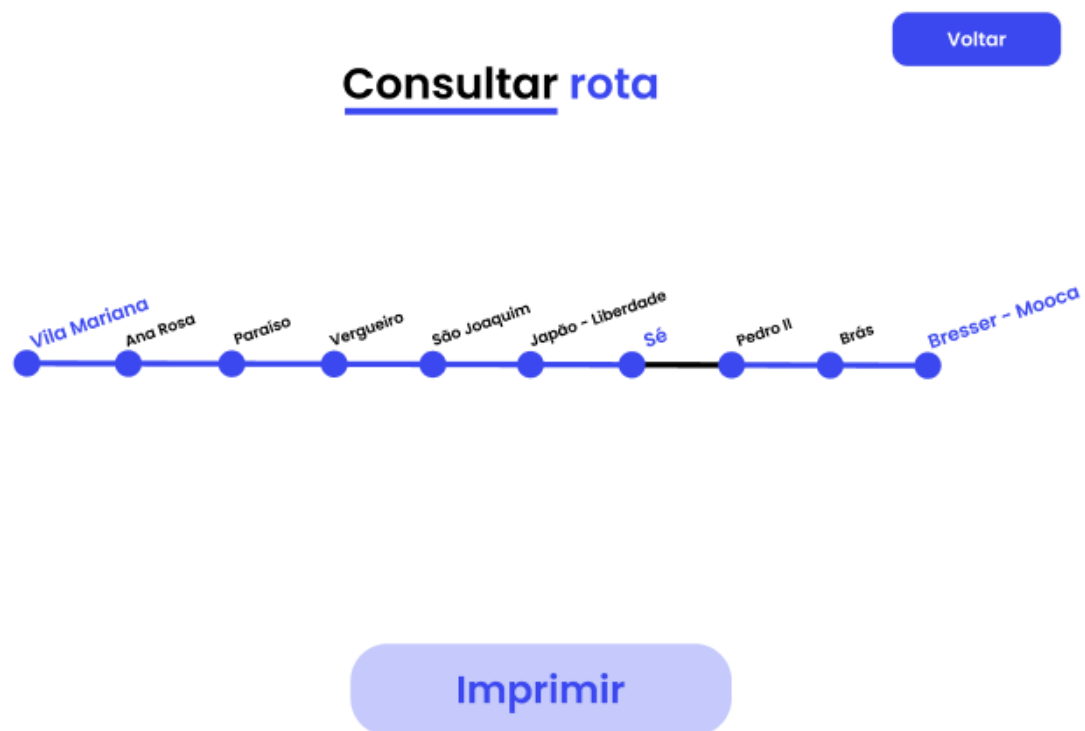
```

1 public static void consultarCaminho() {
2     String origem = JOptionPane.showInputDialog(null, "Qual estação você está?");
3     String destino = JOptionPane.showInputDialog(null, "Qual estação você quer ir?");
4
5     // Criar o grafo e adicionar as arestas
6     Grafo grafo = new Grafo();
7     grafo.addEdge("Júlio Prestes", "Palmeiras-Barra Funda", 1);
8     grafo.addEdge("Palmeiras-Barra Funda", "Júlio Prestes", 1);
9     grafo.addEdge("Palmeiras-Barra Funda", "Lapa", 1);
10    grafo.addEdge("Lapa", "Palmeiras-Barra Funda", 1);
11    grafo.addEdge("Lapa", "Domingos de Moraes", 1);
12    grafo.addEdge("Domingos de Moraes", "Lapa", 1);
13    grafo.addEdge("Domingos de Moraes", "Imperatriz Leopoldina", 1);
14    grafo.addEdge("Imperatriz Leopoldina", "Domingos de Moraes", 1);
15    grafo.addEdge("Imperatriz Leopoldina", "Presidente Altino", 1);
16    grafo.addEdge("Presidente Altino", "Imperatriz Leopoldina", 1);
17    grafo.addEdge("Presidente Altino", "Osasco", 1);
18    grafo.addEdge("Osasco", "Presidente Altino", 1);
19    grafo.addEdge("Osasco", "Comandante Sampaio", 1);
20    grafo.addEdge("Comandante Sampaio", "Osasco", 1);
21    grafo.addEdge("Comandante Sampaio", "Quitaúna", 1);
22    grafo.addEdge("Quitaúna", "Comandante Sampaio", 1);
23    grafo.addEdge("Quitaúna", "General Miguel Costa", 1);
24    grafo.addEdge("General Miguel Costa", "Quitaúna", 1);
25    grafo.addEdge("General Miguel Costa", "Carapicuíba", 1);
26    grafo.addEdge("Carapicuíba", "General Miguel Costa", 1);
27    grafo.addEdge("Carapicuíba", "Santa Terezinha", 1);
28    grafo.addEdge("Santa Terezinha", "Carapicuíba", 1);
29    grafo.addEdge("Santa Terezinha", "Antonio João", 1);
30    grafo.addEdge("Antonio João", "Santa Terezinha", 1);
31    grafo.addEdge("Antonio João", "Barueri", 1);
32    grafo.addEdge("Barueri", "Antonio João", 1);
33    grafo.addEdge("Barueri", "Jardim Belval", 1);
34    grafo.addEdge("Jardim Belval", "Barueri", 1);
35    grafo.addEdge("Jardim Belval", "Jardim Silveira", 1);
36    grafo.addEdge("Jardim Silveira", "Jardim Belval", 1);
37    grafo.addEdge("Jardim Silveira", "Jandira", 1);
38    grafo.addEdge("Jandira", "Jardim Silveira", 1);
39    grafo.addEdge("Jandira", "Sagrado Coração", 1);
40    grafo.addEdge("Sagrado Coração", "Jandira", 1);
41    grafo.addEdge("Sagrado Coração", "Engenheiro Cardoso", 1);
42    grafo.addEdge("Engenheiro Cardoso", "Sagrado Coração", 1);
43    grafo.addEdge("Engenheiro Cardoso", "Itapevi", 1);
44    grafo.addEdge("Itapevi", "Engenheiro Cardoso", 1);
45    grafo.addEdge("Itapevi", "Amador Bueno", 1);
46    grafo.addEdge("Amador Bueno", "Itapevi", 1);
47
48    // Encontrar o caminho mais curto
49    List<String> caminhoMaisCurto = grafo.caminhoMaisCurto(origem, destino);
50
51    // Exibir o caminho
52    if (caminhoMaisCurto.isEmpty()) {
53        JOptionPane.showMessageDialog(null, "Não foi possível encontrar o caminho entre as estações.");
54    } else {
55        JOptionPane.showMessageDialog(null, "Caminho mais curto: " + caminhoMaisCurto);
56    }
57 }

```

O usuário irá digitar a estação de origem e destino na tela do totem, o algoritmo irá receber a informação e assim executará o algoritmo de Dijkstra para encontrar o melhor caminho que usuário deverá percorrer para chegar ao seu destino.

3.Tela Rota



Na tela de rota, será retornado a busca realizada pelo algoritmo de trajeto que o usuário deverá percorrer para chegar em sua estação de destino.

ENPOINTS

URI	Método HTTP	Ação / Descrição	Códigos de Status Possíveis
/duvidas	GET	Retorna todas as dúvidas frequentes	200 OK, 500 Internal Server Error
/duvidas	POST	Cadastra uma nova dúvida frequente	201 Created, 400 Bad Request, 500 Internal
/duvidas/{id}	PUT	Atualiza uma dúvida frequente existente pelo ID	200 OK, 404 Not Found, 500 Internal
/duvidas/{id}/status	PUT	Atualiza o status de uma dúvida (ativo/inativo) pelo ID	200 OK, 404 Not Found, 500 Internal

BANCO DE DADOS

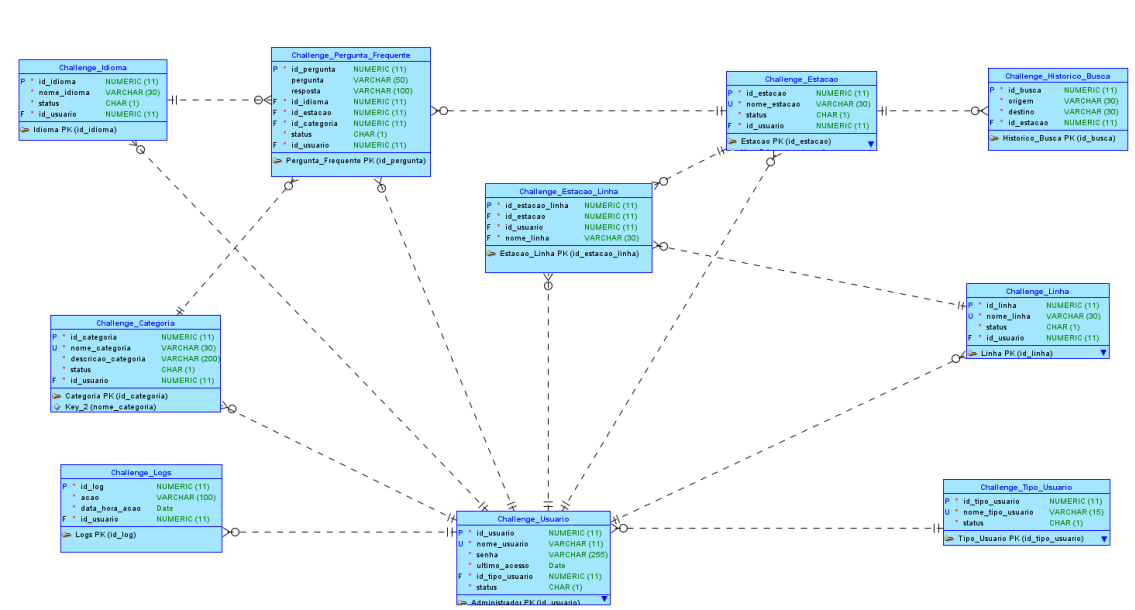
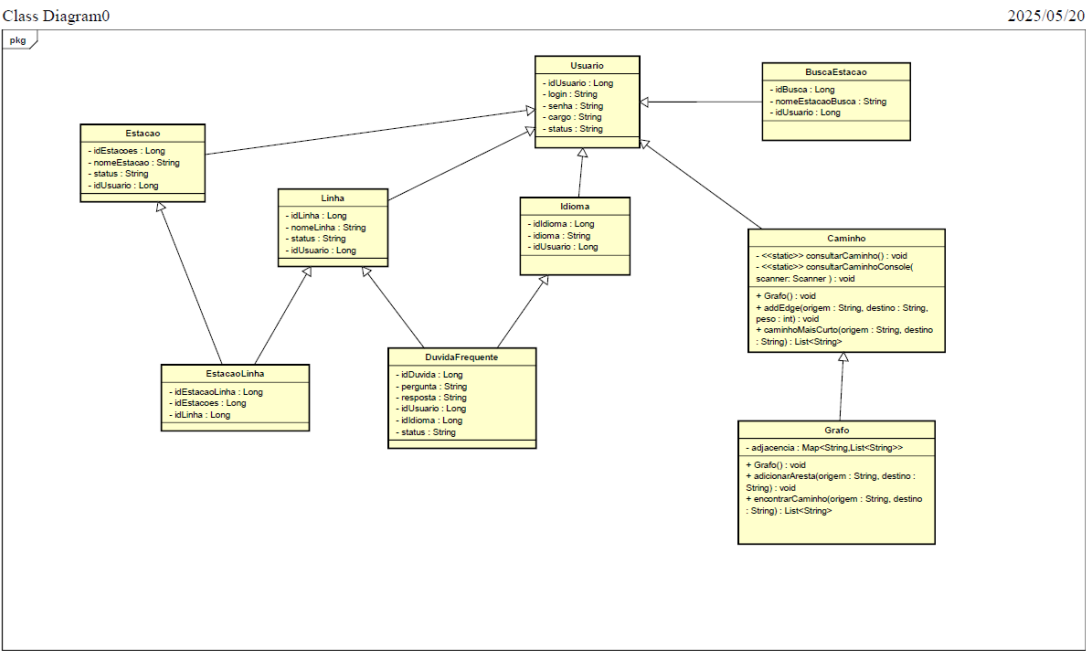


DIAGRAMA DE CLASSE



LINK DO GITHUB DO PROJETO

<https://github.com/MatheusHenriqueNF/Java-Sprint-4>